Q-Day Prize Submission Brief

Author: Giancarlo Lelli | **Contact**: gcarlo.lelli@gmail.com

Overview

This submission implements Shor's algorithm for the Elliptic Curve Discrete Logarithm Problem (ECDLP) on IBM Quantum hardware, targeting the Q-Day Prize challenge curves ($y^2 = x^3 + 7$ over F_p). The solver successfully recovered private keys for challenge curves up to 9-bit on real quantum hardware using only the IBM Quantum open-instance free tier (10 minutes/month).

Algorithm

The solver implements the two-register variant of Shor's algorithm for ECDLP. Given a generator G and public key Q = dG on an elliptic curve of order n, the circuit:

1. Prepares two counting registers |j>, |k> in uniform superposition via Hadamard gates (t = n_bits + 1 qubits each).
2. Applies 2t controlled point additions to compute |j>|k>|jG + kQ> in the point register.
3. Measures the point register, collapsing it to a group element R.
4. Applies inverse QFT to both counting registers.
5. Measures j, k and extracts d from the linear relation j + kd = r (mod n) using modular inversion.

The private key d is recovered by collecting (j, k) samples across 8,192 shots. Both direct extraction (single-shot: $d = (r - j) * k^{-1} \bmod n$) and pair-based extraction (eliminating r between two measurements sharing the same collapsed point) are used to maximize recovery probability from noisy hardware results. No knowledge of d is used in circuit construction. The algorithm is general and applies to any curve in the challenge set.

Circuit Strategies

Dense Unitary (n_bits <= 6, default): Each controlled point addition is a $2^{n+1} \times 2^{n+1}$ permutation matrix encoding the full group action. Applied via `qc.unitary()` and decomposed by Qiskit. Memory: $O(2^{2n})$ per matrix. Used for the 4-bit and 6-bit challenges.

Efficient Permutation Decomposition (n_bits > 6, default): Each "add S" permutation is cycle-decomposed into transpositions. Each transposition swaps two basis states |a> <-> |b> using a CNOT-reduction method: CNOTs from a pivot differing-bit collapse the multi-bit difference to a single bit, then a multi-controlled X (with X-gate conditioning) performs the swap, followed by CNOT reversal. The MCX uses V-chain decomposition with (n-2) dedicated ancilla qubits, achieving O(n) Toffoli gates per MCX instead of $O(n^2)$. Each controlled addition is built as an isolated sub-circuit and appended to the main circuit as a single opaque gate, avoiding quadratic Qiskit DAG growth. Memory: O(N). Used for the 8-bit and 9-bit challenges.

Coordinate-Based Quantum Oracle (--oracle coordinate, n_bits <= 6): Instead of encoding points as group indices, the quantum register holds actual (x, y) field-element coordinates in binary plus an identity flag (1 qubit). The point register width is 2*ceil(log2(p)) + 1 qubits. Each controlled "add classical S" operation is computed from the EC addition formula for every valid coordinate encoding, producing a permutation on the coordinate register that is cycle-decomposed into transpositions and implemented using the same CNOT-reduction + V-chain MCX infrastructure as the efficient permutation strategy. This oracle represents a

fundamentally different encoding: qubits store actual elliptic curve coordinates rather than abstract group indices. Selectable via `--oracle coordinate` for curves up to 6-bit.

Results

All executions ran on IBM Quantum hardware via `qiskit-ibm-runtime` SamplerV2, transpiled at optimization level 3.

Challenge	Prime p	Order n	Strategy	Qubits	2Q Gates	Transpiled Depth	Backend	Recovered d
4-bit	13	7	Dense unitary	11	774	2,425	ibm_torino	**6**
4-bit	13	7	Coordinate oracle	24	6,449	13,125	ibm_kingston	**6**
6-bit	43	31	Dense unitary	17	23,471	72,475	ibm_torino	**18**
6-bit	43	31	Coordinate oracle	36	95,254	169,766	ibm_kingston	**18**
8-bit	163	139	Efficient perm.	32	294,628	599,517	ibm_kingston	**103**
9-bit	349	313	Efficient perm.	36	887,544	1,764,266	ibm_torino	**135**

All runs used 8,192 shots on the IBM Quantum open-instance plan. Every recovered key was verified classically: d * G = Q.

Resource Scaling

Qubit count scales as 2(n+1) + n + max(0, n-2) = 4n for the efficient strategy. Gate count scales as O(t * N * n) where t = n+1 counting qubits, N = group order, and n = bit length. For the 9-bit curve this produced ~887K two-qubit gates after transpilation. The coordinate oracle strategy uses 2(n+1) + 2*f_bits + 1 + max(0, 2*f_bits - 1) qubits, where f_bits = ceil(log2(p)). The wider point register (coordinates vs indices) increases qubit count (24 vs 11 for 4-bit, 36 vs 17 for 6-bit) and gate count, but provides a genuine coordinate-space representation of the elliptic curve arithmetic. Both the 4-bit and 6-bit coordinate oracle runs successfully recovered the correct private key on IBM Quantum hardware. The codebase also includes QFT-based modular arithmetic primitives (Beauregard adders, modular multiplication, Fermat inversion) as building blocks toward a fully arithmetic coordinate-encoding approach that would achieve polynomial O(n^3) gate scaling, necessary for reaching 256-bit keys.

Reproducibility

The solver is fully self-contained. To reproduce:

```
git clone https://github.com/GiancarloLelli/quantum.git
cd quantum && python -m venv . && Scripts/Activate.ps1
pip install -r requirements.txt
python projecteleven.py --challenge 4 --token YOUR_IBM_TOKEN --backend ibm_torino
```

All source code, challenge curves (`input_curves.json`), and execution logs (`executions/`) are included in the repository. The solver requires only `qiskit` and `qiskit-ibm-runtime` -- no local simulator, no classical pre-computation of the secret key.